

1. The Problem: Synchronous vs. Asynchronous Email

✗ The Flawed Synchronous Approach

High Latency & Fragile

When a user registers, if the backend attempts to connect to an external SMTP server inside the signup HTTP request:

- Latency Spike:** SMTP handshakes take 1,500ms – 4,000ms. The user stares at a spinner.
- Hard Failures:** If the SMTP gateway has a transient hiccup, registration crashes with HTTP 500.
- Cascade Outages:** Slow email requests hold backend socket threads open, causing connection pool exhaustion.

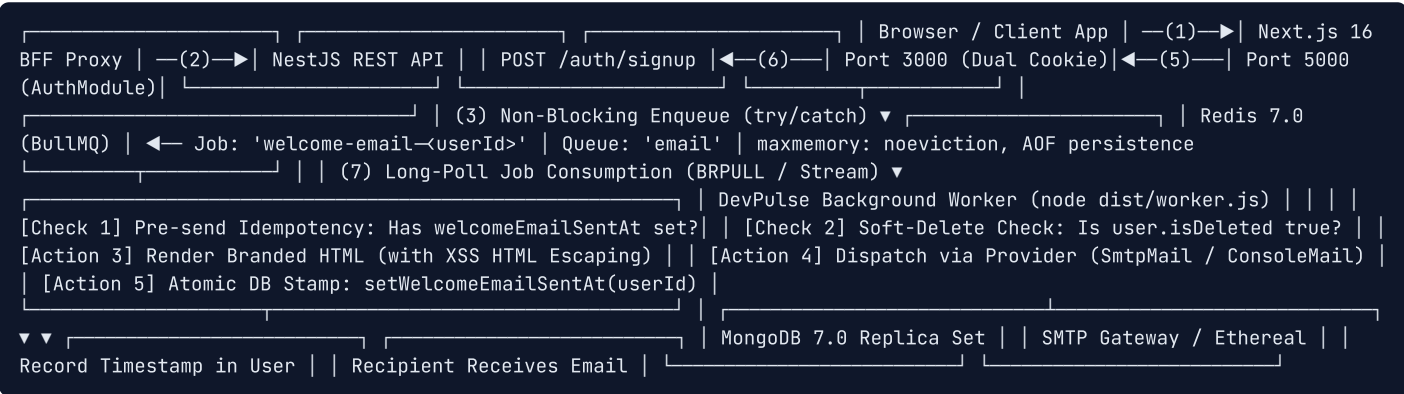
✓ The DevPulse Queue & Worker Solution

Resilient & Sub-20ms

With decoupled message queues, signup is completely isolated from email delivery:

- Instant Response:** API creates the user, pushes a lightweight job to Redis in 2ms, and returns HTTP 201.
- Zero Registration Failures:** If Redis or SMTP is completely down, signup still succeeds seamlessly.
- Autonomous Worker:** A separate background worker picks up jobs, manages retries, and enforces idempotency.

2. High-Level Architectural Pipeline



3. Key Architectural Pillars

Decoupled Processes

Scale

API (main.ts) and Worker (worker.ts) run as completely independent OS processes and Docker containers. API traffic surges never starve background email processing.

Provider Abstraction

Clean Code

Using Dependency Injection (MAIL_PROVIDER_TOKEN), the worker seamlessly switches between ConsoleMailProvider (local dev with masked PII) and SmtpMailProvider (Nodemailer / Ethereal / SendGrid).

Strict Idempotency

Safety

Deterministic Job IDs (welcome-email-`<userId>`) prevent Redis job duplication, and database-level `welcomeEmailSentAt` timestamps prevent duplicate emails if a job is retried.

1. The 10-Step End-to-End Processing Workflow

- 1 User Registration Initiated:** Client submits `POST /auth/signup` with name, email, and password.
- 2 User Record Persisted:** `AuthService` hashes password with `bcrypt` and saves the new User in MongoDB with `welcomeEmailSentAt: null`.
- 3 Non-Blocking Enqueue:** `MailProducerService.enqueueWelcomeEmail()` creates a BullMQ job with deterministic ID `welcome-email-<userId>` and pushes it to Redis. This call is wrapped in a resilient `try/catch` block.
- 4 Immediate Client Response:** The API responds with `HTTP 201 Created` and authentication cookies in less than 20 milliseconds. The user is logged in instantly.
- 5 Worker Picks Up Job:** The standalone `WorkerModule` process listening on Redis pops the job and passes the payload to `MailProcessor.process()`.
- 6 Pre-Send Idempotency & State Check:** Worker queries MongoDB via `UserService.findByIdForEmailProcessing(userId)`. If the user already has a `welcomeEmailSentAt` timestamp, the job is cleanly logged and skipped. If the user is marked `isDeleted: true`, it is skipped.
- 7 Template Compilation:** `buildWelcomeEmailHtml()` renders responsive branded HTML. All user inputs (names/emails) are sanitized through HTML character escaping to eliminate XSS injection risks.
- 8 Transmission via MailProvider:** `MailService.sendWelcomeEmail()` calls the active provider. In production, `SmtplibMailProvider` connects via Nodemailer to the SMTP relay. In local test mode, `ConsoleMailProvider` logs masked recipients (e.g. `a***k@example.com`).
- 9 Atomic Database Timestamping:** Following successful SMTP receipt, the worker invokes `UserService.setWelcomeEmailSentAt(userId, new Date())`, permanently locking the record against re-sends.
- 10 Job Completion & Lifecycle Cleanup:** BullMQ marks the job completed in Redis. BullMQ automatically cleans up the completed job key per the `removeOnComplete: true` policy.

2. Infrastructure Failure & Resilience Matrix

Failure Scenario	System Behavior	Impact on User & Data
Worker Offline During Signup	API enqueues job into Redis normally. Job enters waiting state. Redis persists queue to disk via AOF.	Zero user impact (HTTP 201). When worker restarts, it instantly picks up and processes all waiting jobs.
Redis Down During Signup	<code>AuthService</code> catches the Redis connection error, logs an error alert, and returns <code>HTTP 201 unblocked</code> .	Registration succeeds without delay. System degrades gracefully rather than failing the user.
SMTP Gateway Down / Timeout	Worker detects SMTP error and throws an exception. BullMQ captures failure and schedules automatic retry.	Job enters <code>delayed</code> state. Retried up to 3 times with exponential backoff (5s, 10s, 20s).
Worker Crash Mid-Transmission	BullMQ detects worker stall/heartbeat loss via lock renewal timeouts and re-assigns job to active worker.	At-least-once delivery: worker re-executes. Pre-send idempotency guard prevents double-send if DB write succeeded.
User Deleted Before Send	Worker queries user in MongoDB, observes <code>isDeleted: true</code> flag, logs warning, and completes.	Compliance preserved: no emails sent to soft-deleted or removed user accounts.

3. Database Idempotency Field Specifications

Mongoose User Schema: `welcomeEmailSentAt`

Schema Safety

To prevent accidental data leakage, the field is defined with `select: false` in Mongoose and explicitly stripped in `schema.set('toJSON')`. It is never exposed over public REST API endpoints or frontend user profile queries. It is accessed exclusively by internal worker queries.

1. Resilient Producer: AuthService & MailProducerService

In backend/src/auth/auth.service.ts, signup never halts if Redis is unavailable:

```
// backend/src/auth/auth.service.ts (signup method) try { await this.mailProducerService.enqueueWelcomeEmail({ userId: user._id.toString(), email: user.email, name: user.name, }); } catch (error) { this.logger.error(`Failed to enqueue welcome email for user ${user._id}: ${error.message}`); // IMPORTANT: Do NOT re-throw! User signup must proceed smoothly with 201 Created. }
```

In backend/src/mail-queue/mail-producer.service.ts, deterministic job IDs prevent double queueing:

```
// backend/src/mail-queue/mail-producer.service.ts const jobId = `welcome-email-${payload.userId}`; await this.emailQueue.add(WELCOME_EMAIL_JOB, payload, { jobId, // Dedupes duplicate requests for the exact same user attempts: 3, // Retry 3 times if SMTP fails backoff: { type: 'exponential', delay: 5000 }, // 5s, 10s, 20s backoff removeOnComplete: true, // Purge key on success to keep Redis lean removeOnFail: false, // Retain failed job in dead-letter set for inspection });
```

2. Consumer & Idempotency: MailProcessor

In backend/src/mail-queue/mail.processor.ts, worker checks the database before sending:

```
// backend/src/mail-queue/mail.processor.ts (@Processor(MAIL_QUEUE_NAME)) const user = await this.userService.findByIdForEmailProcessing(userId); if (!user) return; // User no longer exists if (user.isDeleted) return; // Soft-deleted user: skip if (user.welcomeEmailSentAt) { this.logger.log(`User already received email at ${user.welcomeEmailSentAt}. Skipping.`); return; // Idempotency guard: never send twice! } // 1. Send email via active MailProvider await this.mailService.sendWelcomeEmail({ to: user.email, name: user.name }); // 2. Record timestamp immediately upon delivery await this.userService.setWelcomeEmailSentAt(userId, new Date());
```

3. Dedicated Worker Entrypoint: worker.ts

In backend/src/worker.ts, worker boots as an independent Nest application context:

```
// backend/src/worker.ts const app = await NestFactory.createApplicationContext(WorkerModule); app.enableShutdownHooks(); // Cleanly closes Redis connections on SIGTERM/SIGINT logger.log('✅ DevPulse Email Worker is active and listening for queue jobs.');
```

1. Live Smoke Test Suite (Step 7 Verification)

Executed against real Redis container and live Ethereum SMTP server via backend/test/smoke-test-step7.mjs:

Scenario	Condition Tested	Observed Live Result	Status
Scenario 1	Full End-to-End Pipeline	Signup 201 → BullMQ job active → Ethereum SMTP delivered → DB timestamp set (2026-10-08T08:17:11.610Z).	PASSED
Scenario 2	Worker Offline Queuing	Worker stopped → Signup 201 → Job waited in Redis (state: waiting) → Worker started → Job completed.	PASSED
Scenario 3	Redis Outage Resilience	Docker stopped Redis → Signup submitted → API returned HTTP 201 (unblocked) → Redis restarted → Job drained.	PASSED
Scenario 4	SMTP Failure Retry	Worker with invalid SMTP port → Transmission failed → BullMQ transitioned job to delayed (attemptsMade: 1).	PASSED

2. Docker Compose Multi-Container Orchestration

```
# docker-compose.yml additions
redis:
  image: redis:7-alpine
  command: redis-server --requirepass devpulse_redis_secret
  --maxmemory-policy noeviction --appendonly yes
  ports: ["6379:6379"]
  volumes: [redis_data:/data]
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "devpulse_redis_secret", "ping"]
  worker:
    build: { context: ./backend, dockerfile: Dockerfile }
    command: node dist/worker.js
    restart: unless-stopped
    depends_on:
      redis: { condition: service_healthy }
    mongo: { condition: service_healthy }
```

3. Developer Quick Reference & Commands

Local Development

Host Run

Run API and Worker concurrently during development:

npm run start:dev # Terminal 1: API (5000)

npm run start:worker:dev # Terminal 2: Worker

Automated Testing

Vitest

All 34 test files and 240 tests pass with 100% integrity:

npm run test # Run all 240 tests

node test/smoke-test-step7.mjs # Live 4-scenario E2E

Engineering Summary: The welcome email infrastructure decouples business actions from transport mechanics, ensuring high-availability signups, deterministic job execution, at-least-once delivery safety, and complete operational transparency.